

Twelve expert briefings, written and shipped every night by a crew of agents that never meet

A working tour of how a standing prompt becomes a researched, fact-audited, narrated podcast episode on Spotify — the five agents, the artifacts they hand each other, the deterministic gate none of them can talk their way past, and everything that happens after the last word is written.

REPO · [ClaudeDev/Spotify](#) FEED · [marange63.github.io/Spotify/feed.xml](#)

SHOW · Cautious Optimism Briefings REV · 2026-07-25

01 What the machine makes

Every night, without a human in the loop, the system turns a library of standing topic prompts into a batch of fresh audio briefings and publishes them as new episodes of a public podcast.

The listener is assumed to be fluent in economics, capital markets, technology and AI. That assumption is the product: no 101-level definitions, no recaps of well-known context, lead with what is genuinely new or non-consensus, and carry analysis rather than headlines. A briefing that has nothing new to say is **skipped** rather than padded — a skipped day beats a weak episode, and the architecture is built to make skipping easy and cheap.

One prompt produces one episode. Twelve prompts run per night, and a thirteenth kind of prompt — **The Throughline** — is a **synthesis** that reads the other twelve after they are finished and digests them into a front page. The whole batch publishes together as new episodes with per-day GUIDs, so followers get normal new-episode notifications.

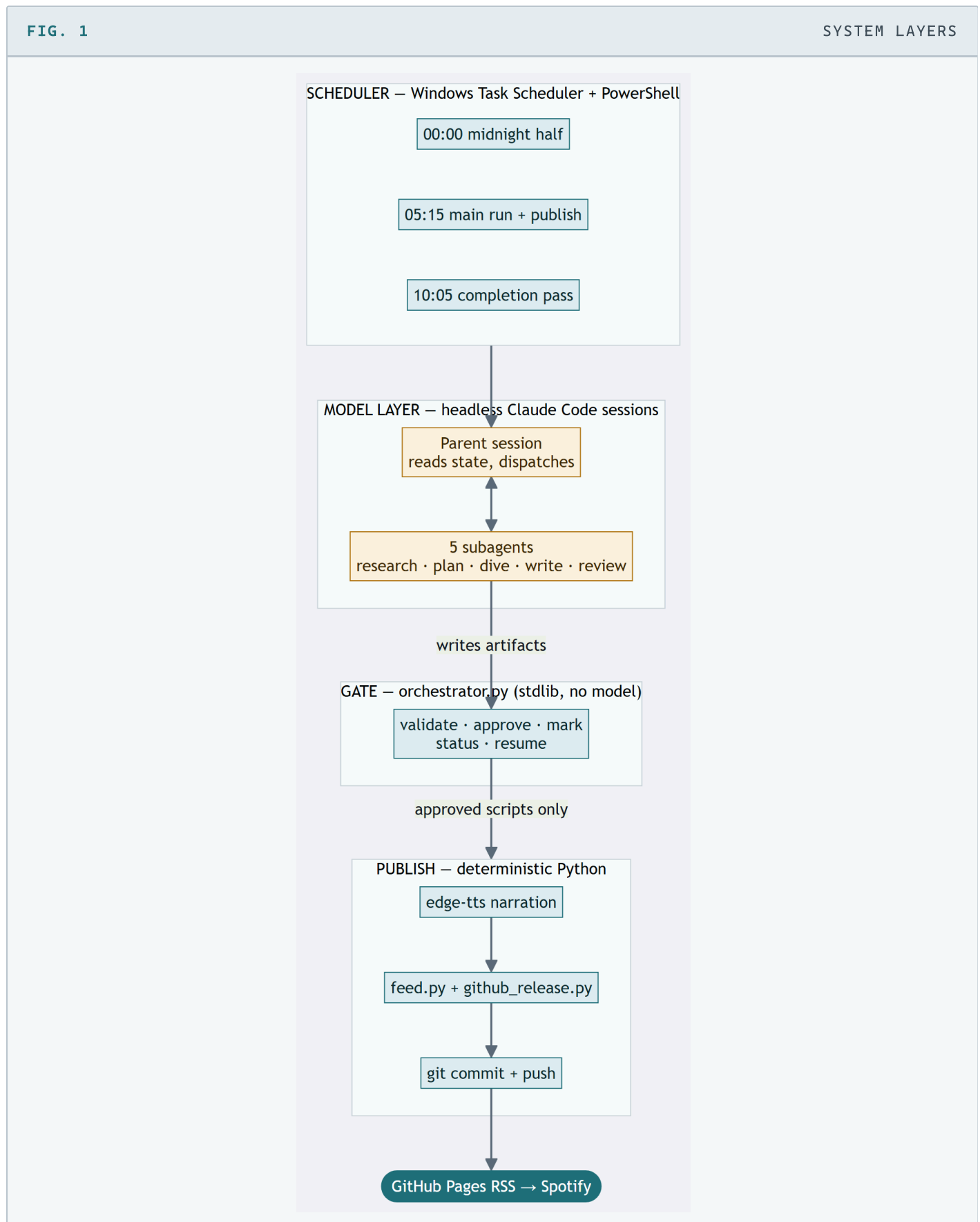
02 The governing idea: agents write, code decides

Everything else in this document follows from one split. **Language models produce judgment and prose. Plain Python decides what is allowed to happen.** No agent can publish, commit, push, or promote its own work — those verbs live exclusively in deterministic code that reads the agents' output files and applies fixed rules to them.

Concretely: an agent's entire influence on the outside world is the *file it writes to disk*. A gatekeeper (`orchestrator.py`, stdlib only, no model calls) schema-checks that file and refuses to advance the prompt if it fails. The only path from a written script to a published episode is `orchestrator.py approve`, which copies a finished script into `briefings/` *only* if a separate reviewer agent's verdict file says `approve`. Everything downstream — audio, feed, git, Spotify — is triggered by a second, entirely non-AI phase.

Throughout the diagrams that follow, **amber** marks work done by a model and **teal** marks deterministic code.

Figure 1 – Three layers. The scheduler starts things, models do the thinking, and deterministic Python owns every irreversible action.



WHY BOTHER

An agent that can publish can publish a mistake. By making the gate a stdlib program with no model in it, the failure modes become inspectable: a malformed dossier fails a schema check, a rejected draft fails an equality test against the string "approve", and a stale script fails a filemtime comparison. None of those depend on a model behaving well on a bad night.

03 The cast

Five subagents, each defined by a markdown file in `.claude/agents/` whose frontmatter pins its model and its tool access. Those pins **override the caller** — the scheduled run launches a Sonnet parent session, but the analyst and reviewer still run on Opus because their own definitions say so. To change a role's cost or quality you edit that role's frontmatter, never the caller.

Note the tool columns carefully: **only two of the five agents can reach the internet**. That is a deliberate constraint, not an oversight — see §05.

Researcher

STAGE 1

sonnet

web access

Gathers the strongest raw material for one prompt. Surveys the topic, finds what is new, and files a structured dossier. Explicitly does *not* choose the thesis or judge novelty.

```
reads prompt.txt
writes research.json
```

Analyst-Editor

STAGE 2

opus

no web

The editorial brain. Judges the dossier against the prior briefing and the last five days of transcripts, then decides **write or skip**, fixes the thesis, picks and orders the stories, and names the arguments the script must make.

```
reads research.json, prior briefings
writes editorial_plan.json
```

Deep Researcher

STAGE 2.5 · OPTIONAL

sonnet

web access

Runs only when the plan names specific evidence gaps. Answers up to 3 questions on exactly 1 approved story within ~6 web calls. Cannot propose new stories or reopen the plan.

```
reads research.json, editorial_plan.json
writes deep_research.json
```

Writer

STAGE 3

sonnet

no web

Turns plan plus evidence into a spoken script for the ear — one spine, signposted transitions, one idea per sentence. May use **only** figures that carry a verbatim quote in its input files.

```
reads plan + dossier (+ deep dive)
writes draft.txt
```

Reviewer

STAGE 4

opus

no web

An independent editor in a **fresh context** that did not write the draft. Audits every figure against its source quote, critiques against the plan and the standard, revises once, and may refuse. Approve is not its default.

reads draft + all evidence

writes review.json, final.txt

orchestrator.py

THE GATE

stdlib python

no model

Not an agent. Creates the run, schema-checks every artifact, records outcomes, computes where to resume, and owns the single write path into **briefings/**. It is the only component that can say no and mean it.

reads every artifact

writes run.json, briefings/<id>.txt

04 One briefing, end to end

The five agents never talk to each other. They communicate exclusively through **files on disk** under `runs/<date>/<prompt_id>/`, and between every pair of stages the gate validates the handoff. A stage receives no conversational history from the stage before it — only the artifact.

That is what makes the pipeline auditable. Months later you can open one directory and reconstruct exactly why an episode said what it said, or why it was skipped.

Figure 2a – Research and the editorial decision. The dossier is validated before the Analyst-Editor sees it, and the plan may end the prompt outright.

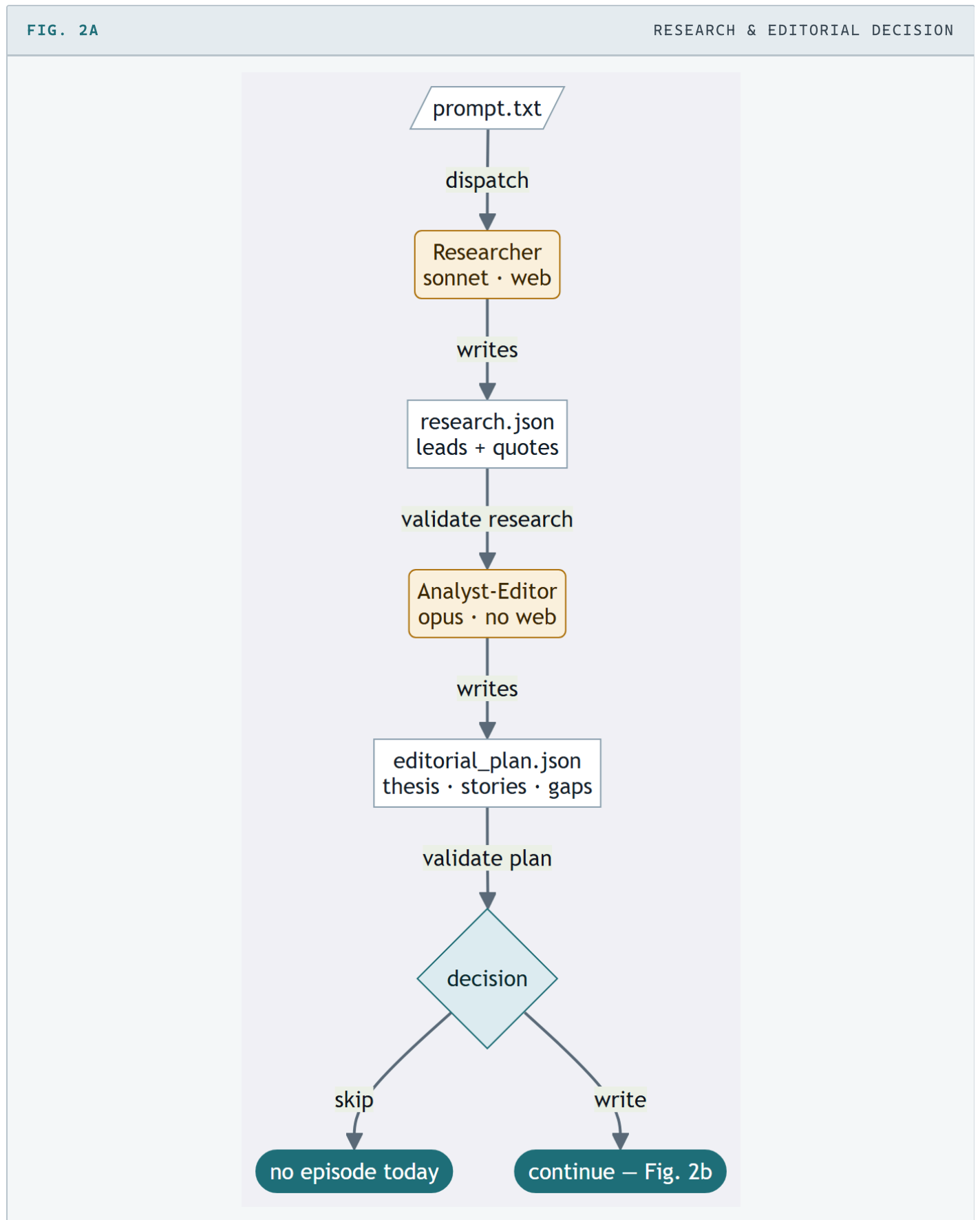
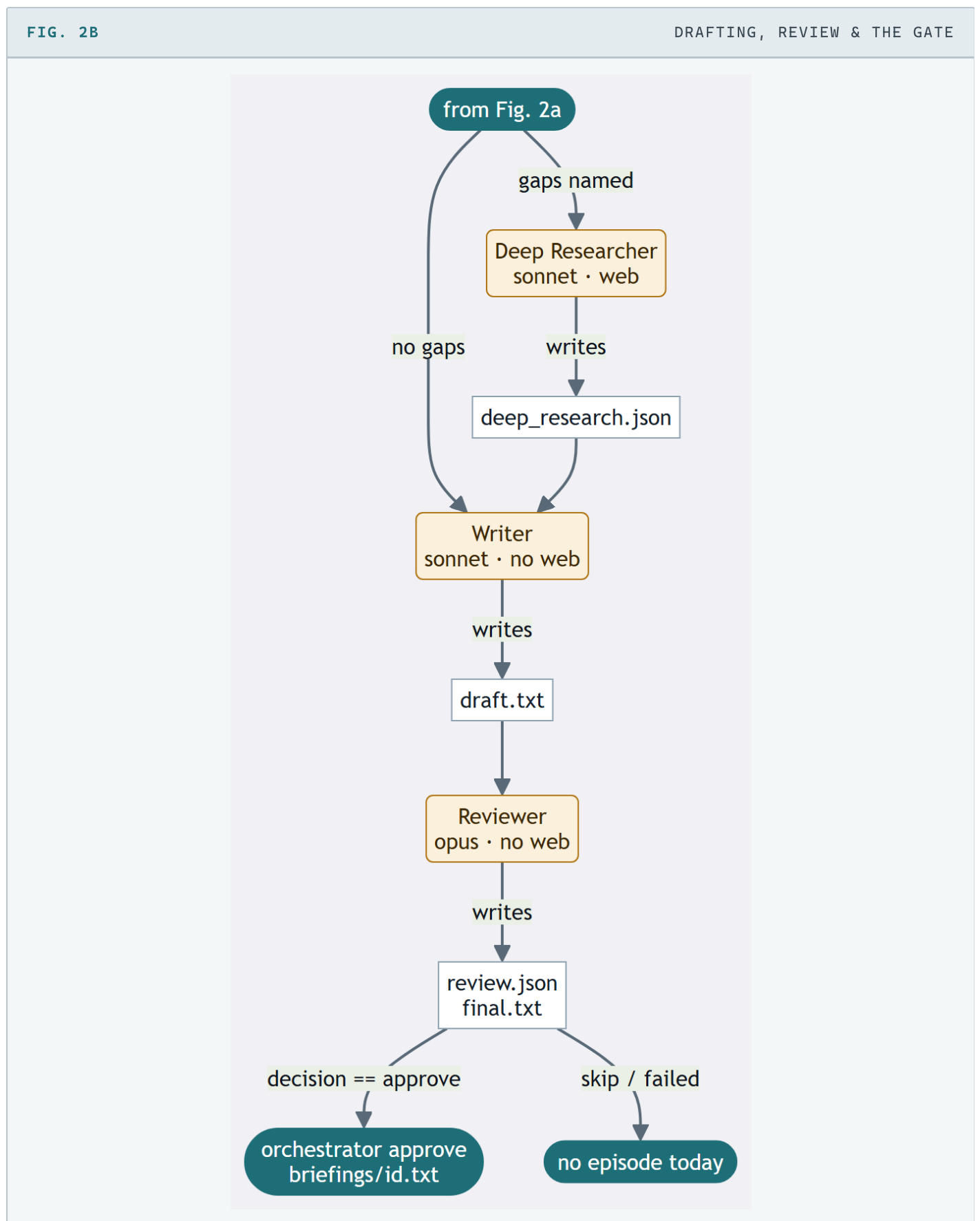


Figure 2b – Evidence, drafting and the gate. The optional deep dive runs only when the plan named gaps; the reviewer holds an independent veto.



The artifact chain

Each row is a file, and the file *is* the interface. Anything not in the file does not reach the next stage.

ARTIFACT	WRITTEN BY	CARRIES	GATE
prompt.txt	orchestrator init	The standing prompt text, written to disk once so agents read it rather than the parent re-sending it every dispatch	–
research.json	Researcher	Lead candidates, secondary items, items deliberately ignored, research gaps – every important fact bound to a verbatim quote	schema + quote contract
editorial_plan.json	Analyst-Editor	decision, thesis, lead story, approved items with treatment, required arguments and second-order effects, emergent 5-day patterns, deep_dive_requests	schema + dive caps
deep_research.json	Deep Researcher	Answers to the named gaps, plus a contradictions array when fresh evidence undercuts the plan. Absent on most prompts.	dossier schema
draft.txt	Writer	The spoken script, first pass	judged, not schema-checked
review.json	Reviewer	decision, issues found, changes made, per-axis scores	schema + decision
final.txt	Reviewer	The revised, publishable script	non-empty

ARTIFACT	WRITTEN BY	CARRIES	GATE
briefings/<id>.txt	orchestrator approve	The copy that TTS and the transcript are built from	only write path

05

Why the seams are where they are

The stage boundaries look bureaucratic until you notice each one removes a specific, observed failure.

The writer is not the reviewer

They are separate agents with separate contexts, and the reviewer is told it did not write the draft. A model grading its own prose approves it; a model handed an anonymous draft and an evidence file audits it. On a recent run the reviewers caught and fixed five figure-audit failures on each of two briefings and three in the synthesis — including a cross-day claim that the last five transcripts actually contradicted.

Three of five agents have no internet

The analyst, writer and reviewer are sealed off from the web on purpose. If the writer could search, it would quietly introduce facts that never passed the researcher's sourcing discipline, and the reviewer would have no fixed evidence set to audit against. Cutting web access converts "is this true?" — unanswerable — into "is this in the dossier?" — mechanically checkable.

Every figure must trace to a verbatim quote

The researcher must attach a literal source `quote` to each important fact, and the schema check enforces it. The writer may use only quoted figures. The reviewer audits each number in the draft back to its quote. A number with no quote gets hedged or cut — which is why the pipeline sometimes says "reported near \$17.9 billion" instead of asserting it.

The plan can say "skip"

Under **strict** novelty — the mode every scheduled run uses — the analyst rejects material that merely repeats the prior briefing unless there is genuinely new news or data. If nothing clears the bar, the prompt is skipped and no episode ships. Interactive runs default to **relaxed**. The reviewer holds a second, independent skip veto at stage 4.

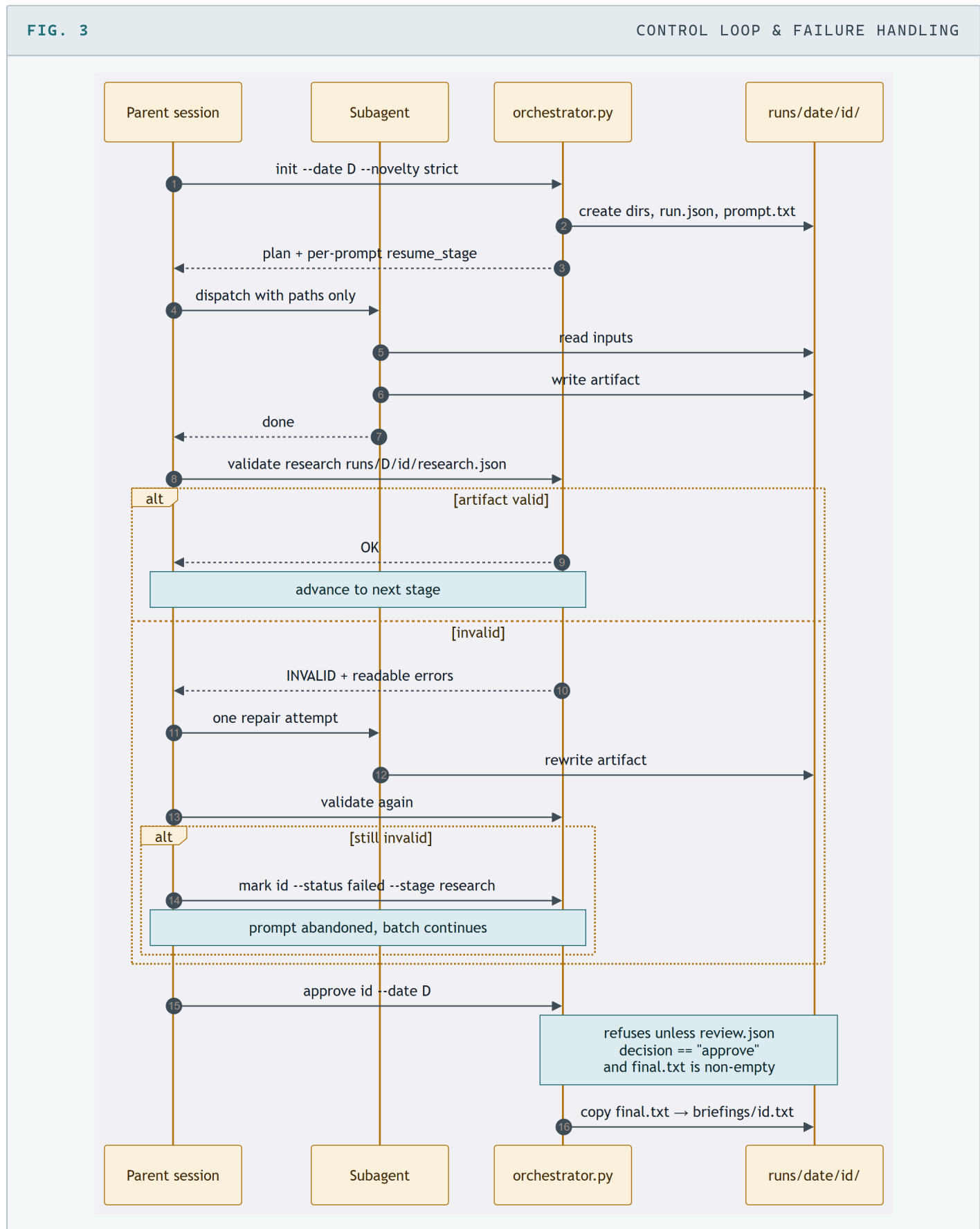
STAGE 2.5 EXISTS TO STOP THE WRITER HEDGING

The analyst writes required arguments knowing the writer has no web access and may only use quoted figures. When the dossier cannot support what the plan demanded, drafts came in short and hedged — 6 of 10 under the word floor on one measured run. The deep dive closes that loop at the one moment the gap is both known and specific, and it is deliberately capped at 1 story, 3 questions, ~6 web calls so a research stage cannot blow the batch budget. It can never fail a prompt: on error its file is deleted and the writer proceeds without it.

06 The gate in motion

Here is the actual control loop for one stage. The parent session never trusts a subagent's word that it succeeded — it validates the file. A malformed artifact gets exactly **one** repair attempt before the prompt is marked failed and the batch moves on. One bad prompt never stops the other eleven.

Figure 3 – Dispatch, validate, repair once, then advance or abandon. The gate's refusal is unconditional: no reviewer verdict, no copy into briefings/.

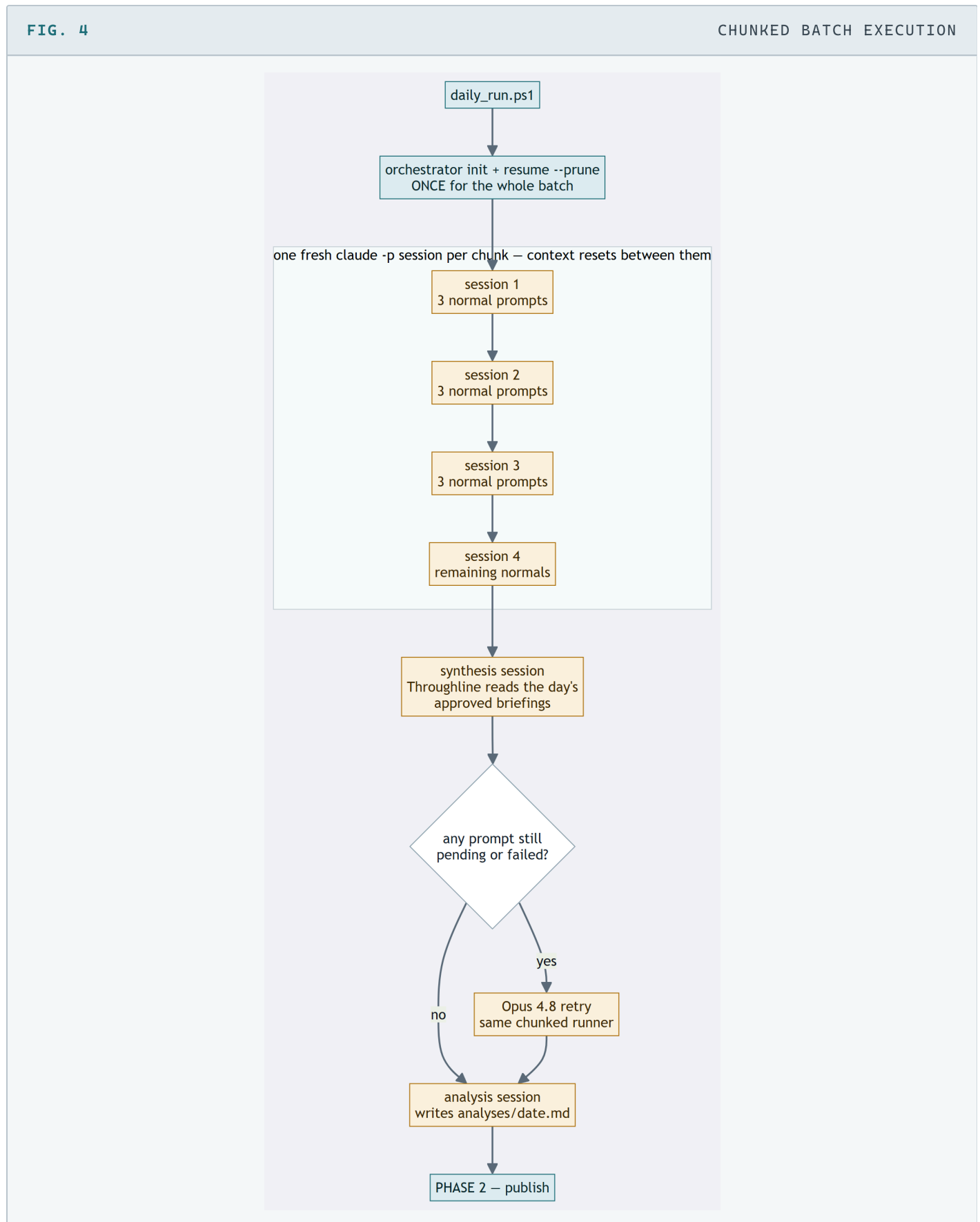


07 From one prompt to a whole batch

Running twelve prompts through five stages inside a single Claude session used to be the design. It failed for an unobvious reason: the parent session's context accumulates across the entire batch, so by prompt twelve every dispatch is re-sending an enormous history. Measured on a real run, that *orchestration* overhead was **36.5% of all tokens spent** — as much as the researcher itself.

Phase 1 therefore runs as **several small sessions**. Prompts are chunked (default 3 per session), each chunk gets a fresh `claude -p` process, and the parent context resets between them. Any synthesis prompt runs **last, alone**, because it must read the day's already-approved briefings off disk. The day's analysis is written in its own final session.

Figure 4 – Chunked phase 1. init and prune run once for the whole batch; each chunk session is told to skip them so a per-chunk prune can never disturb a sibling.



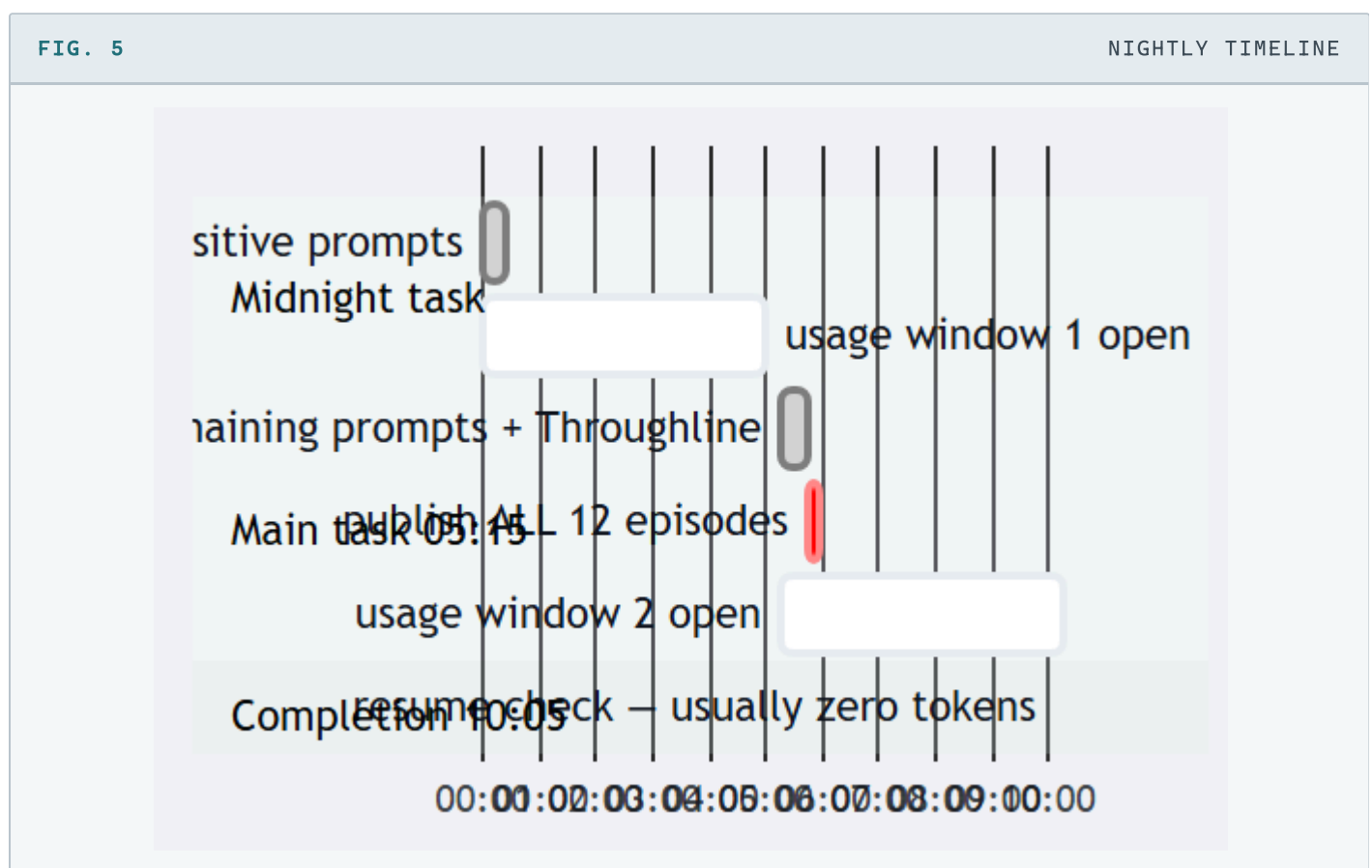
Every prompt is independent, so a chunk that dies takes only its own prompts with it — and even those survive, because of the resume machinery in §09.

08 The night: three runs, two usage windows

Claude usage limits are enforced per rolling ~5-hour **session window**. One full batch now costs more than a single window allows — on 2026-07-25 phase 1 spent ~58M tokens in 16 minutes, finished 9 of 11 prompts, and hit the cap.

Idle time inside a window restores nothing, so pausing mid-run does not help. Starting *earlier* is actively worse: the evening's own interactive session may still hold an open window past midnight. The only pause that helps is one that **crosses the reset boundary**. So the batch is deliberately split across two windows, with a third run as a safety net.

Figure 5 – The nightly schedule. Each run opens its own usage window; the gaps between them are what make the split work.



Why 05:15 and not 05:00

The midnight run's window opens at its first model call and resets ~05:00. Starting the second run at exactly 05:00 lands on the boundary and risks inheriting the spent window. Fifteen minutes buys margin, at the cost of publishing around 05:55.

The midnight half is chosen editorially, not arbitrarily

Its research is ~5 hours staler by air time, so it takes the prompts where that costs least — biopharma, robotics, enterprise tech, private equity, AI products, semiconductors. The market-sensitive briefings (capital markets,

digital money, strategic power, private credit), plus frontier AI labs and the Throughline, stay on the 05:15 run.

Only one publish

The midnight run pipelines its half and stops — it publishes nothing. The 05:15 run picks up everything still outstanding and publishes the **whole day in one commit, one push, one phone notification**. It needs no list of what midnight did: the resume check reports those prompts as `done` and skips them. A newly added prompt therefore falls to the 05:15 run by default, which is the safe side.

The 10:05 completion pass

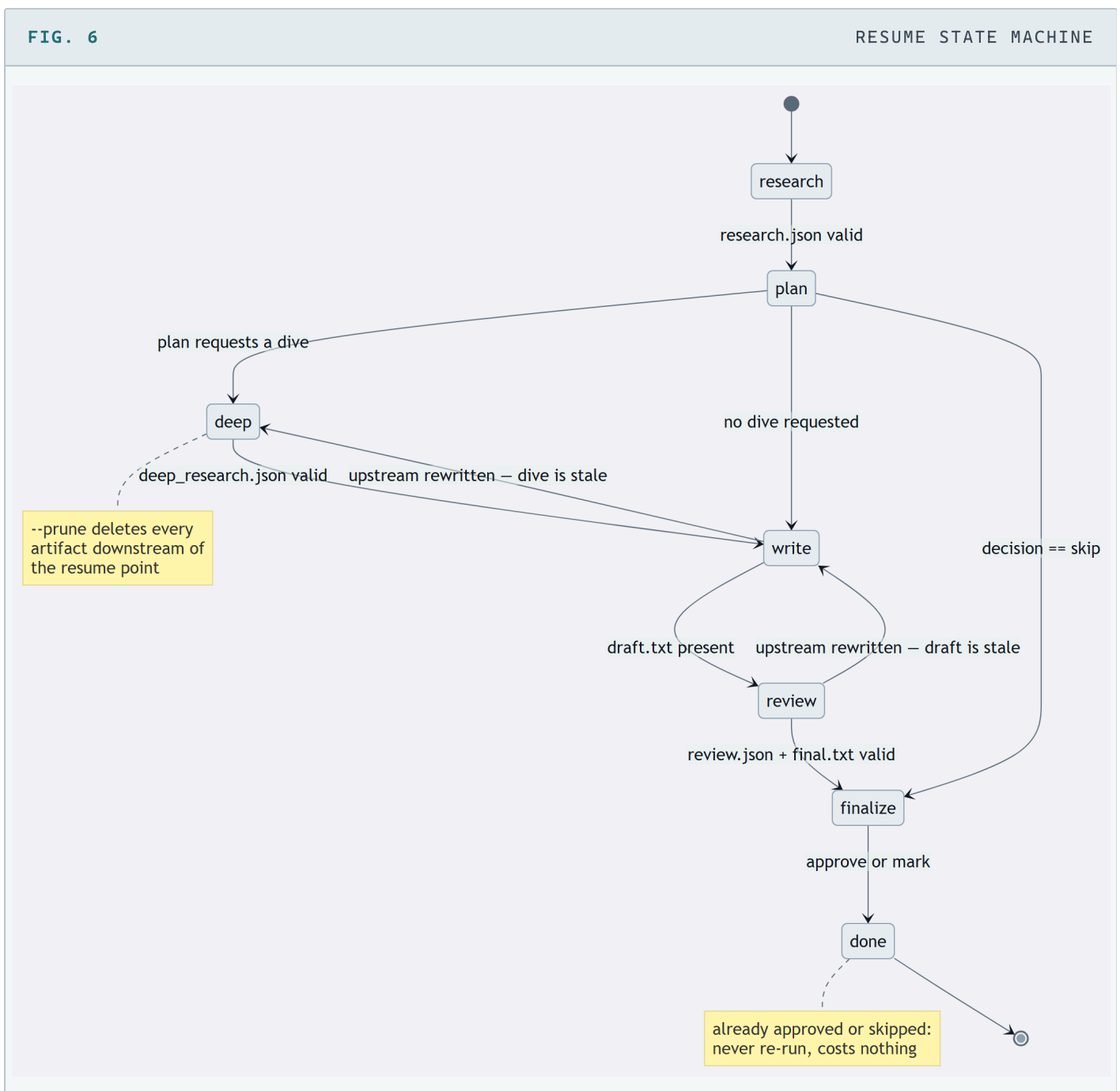
A third scheduled run, just after the 05:15 window resets, finishes anything the cap truncated — in fresh quota, automatically. On a normal morning it asks the resume machinery what is outstanding, finds nothing, **spends zero model tokens**, and exits in about two seconds after confirming every episode is already live.

09 When a run is interrupted

Because agents hand off through files, an interrupted run leaves a partially built directory rather than a lost conversation. `orchestrator.py resume` reads those directories and computes, per prompt, the first stage that actually needs to run.

It walks the artifact chain and stops at the first file that is missing, empty, schema-invalid, or — the subtle one — **older than the artifact it derives from**. Everything from that point onward is stale by construction and is reported for deletion; `--prune` removes it.

Figure 6 – Resume states. Forward edges are normal progress; the backward edges are the staleness rule kicking a prompt back when an upstream artifact was rewritten underneath it.



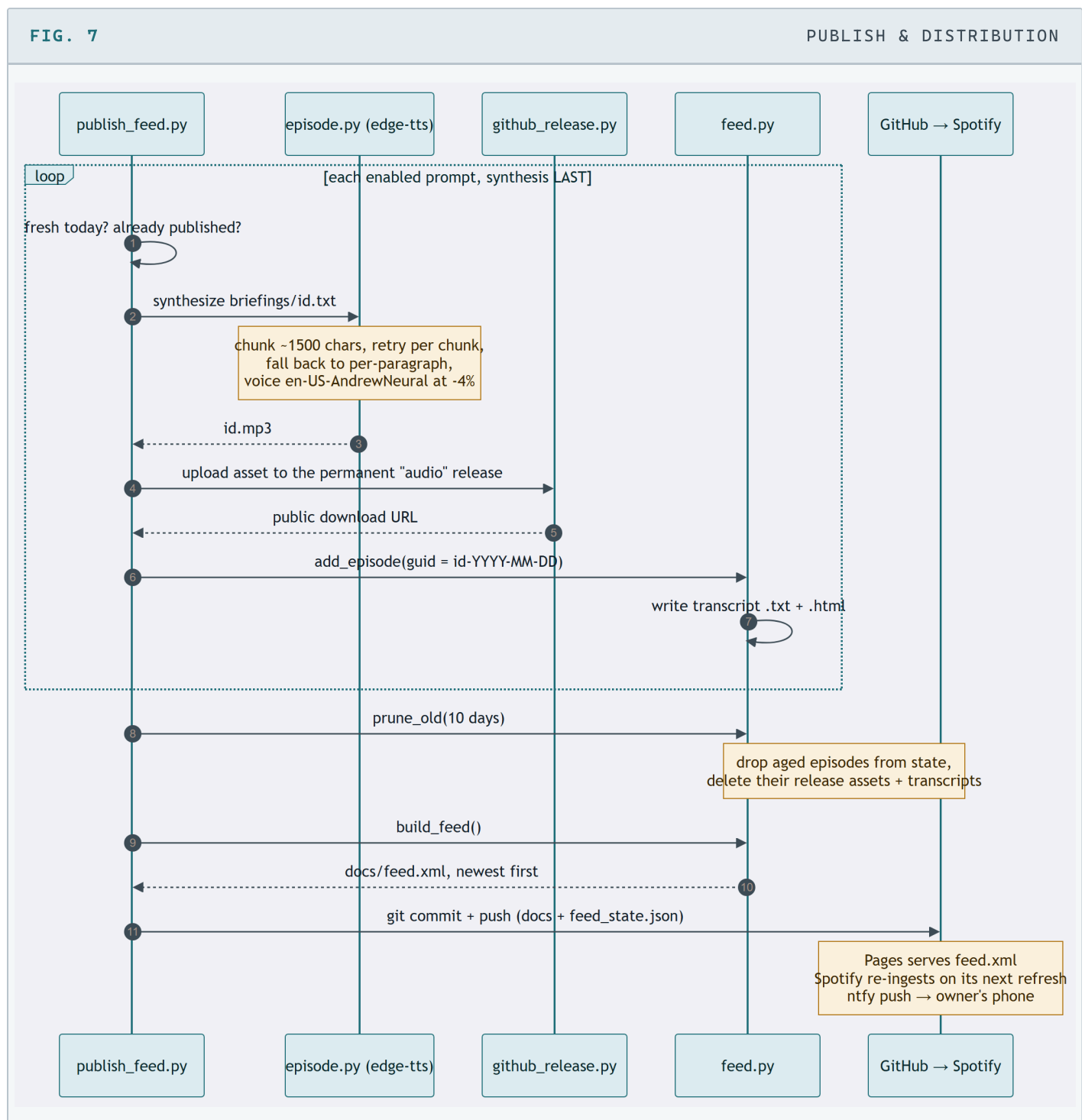
THE FAILURE THIS WAS BUILT FOR

On 2026-07-25 the cap stopped the run mid-batch and the retry restarted each unfinished prompt *from the researcher*, rewriting complete artifacts and exhausting the remaining budget without finishing anything. Worse, the downstream files built from the *old* plan survived on disk with nothing flagging the mismatch — a deep dive about Bitcoin whale flows sitting beside a plan about the CLARITY Act. A naive resume would have handed the writer that mismatched pair, and the resulting script would have looked entirely plausible. Hence: staleness is detected by comparing modification times against the upstream artifact, and pruning is what makes resuming safe.

10 Publishing: from approved script to Spotify

Phase 2 contains no AI at all. It is a deterministic Python batch that takes whatever is sitting in `briefings/` and turns it into a live podcast episode. Its guard rail is `--require-fresh`: only scripts whose file was *modified today* are published, so a prompt that was skipped or failed keeps yesterday's file on disk and is automatically excluded rather than re-shipped as new.

Figure 7 – The publish path. Audio is hosted as GitHub Release assets so the git object store never grows; only the RSS and transcripts live in the repo.



Identity and idempotency

An episode's GUID is `<prompt_id><YYYY-MM-DD>` — unique per topic per day. Re-running the same prompt on the same date overwrites that day's episode in place; a new date creates a new episode. Enclosure URLs carry a `?v=<publish-epoch>` cache-bust token, because replacing already-ingested audio at a stable URL makes Spotify serve the *old* cached file.

Publishing twice in one day

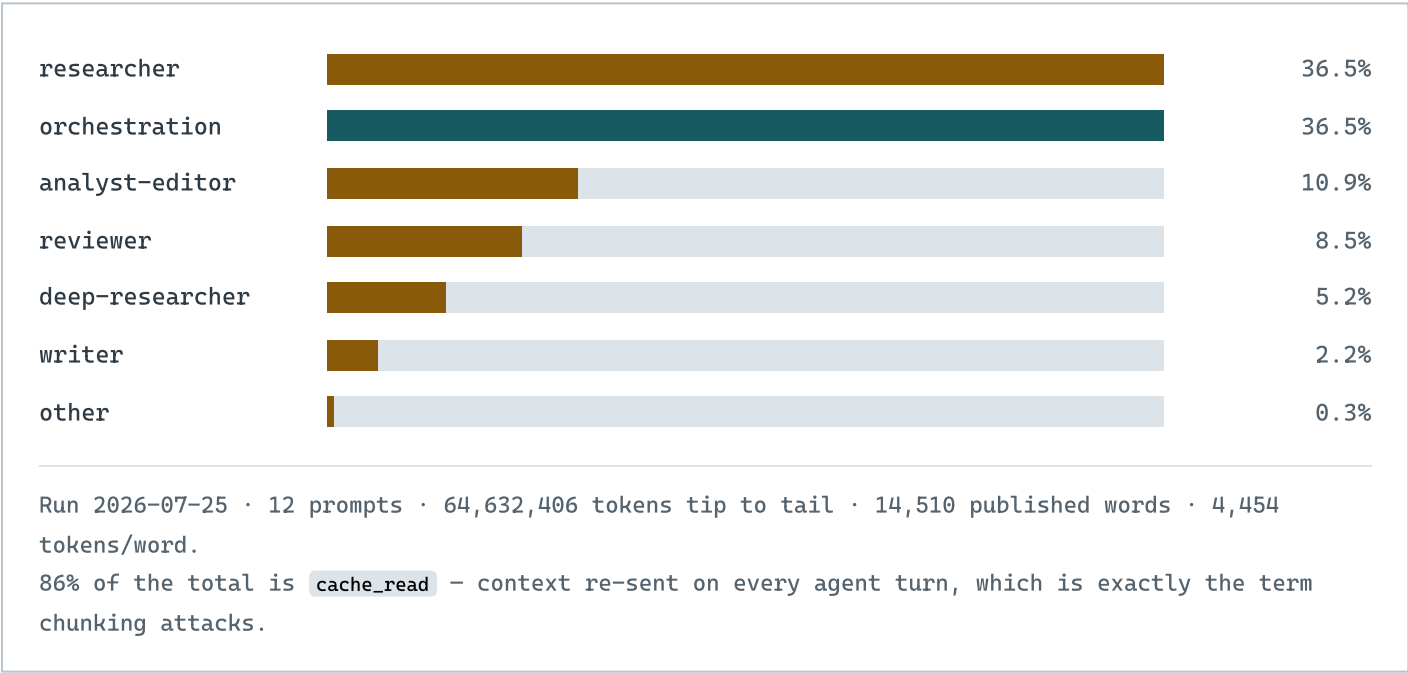
The completion pass adds `--skip-published`, which skips any prompt already recorded in the feed for that date. Without it, a second pass re-runs TTS on live episodes and changes their enclosure URLs, forcing Spotify to re-download audio that never changed. "Already published" counts as success for the exit code but not as newly published, so the phone notification stays silent unless something genuinely shipped.

Rolling retention

The show keeps a **10-day window**. After each publish, aged episodes are dropped from the feed, their audio assets deleted from the GitHub Release, and their transcripts removed; local `runs/` and logs older than the window are swept too. The per-run analyses are deliberately exempt — they are the kept history.

11 What it costs, and where

Every run's token spend is attributed by stage — each subagent identified by the artifact it writes, the parent session booked as *orchestration*. This is the measurement that drives design decisions rather than intuition. Figures below are one real 12-prompt run.



Two findings are worth internalising. First, **writing is nearly free** — the writer is 2.2% of spend. Research and orchestration are the entire cost centre, which is why the levers are chunk size and researcher breadth, not prose quality. Second, **raw token totals are a poor proxy for quota consumption**: an interactive window on 2026-07-24 reached 78.6M tokens without hitting a cap, while a 58M Opus-heavy pipeline run did. Model mix matters more than volume, so the robust move is halving demand per window rather than optimising toward a threshold that cannot be observed.

12 Operator's quick reference

Where to look, in the order worth looking.

QUESTION	WHERE TO LOOK
How did last night go?	<code>logs/daily-<date>.log</code> – all three runs append to one file
What shipped, what skipped?	<code>python orchestrator.py status --date <date></code>
What still needs running?	<code>python orchestrator.py resume --date <date></code> (add <code>--prune</code> before resuming)
Why did this episode say that?	<code>runs/<date>/<id>/</code> – dossier, plan, dive, draft, review
Why was this prompt skipped?	<code>editorial_plan.json</code> → <code>decision_reason</code> , or <code>review.json</code> → <code>decision_reason</code>
How did the agents perform?	<code>analyses/<date>.md</code> , or the Run analyses tab in <code>main.py</code>
Where did the tokens go?	<code>python run_report.py --date <date></code>
Is it live?	<code>feed_state.json</code> , then the public <code>docs/feed.xml</code>
Change a role's model	<code>.claude/agents/<role>.md</code> frontmatter – never the caller
Change what a briefing covers	<code>prompts.json</code> , via the <code>main.py</code> window

The three failure classes

- **One prompt fails.** Marked failed, batch continues, eleven episodes still ship. The stale-script guard keeps the failed one out of the feed.
- **A run is truncated** (usage cap, machine sleep). Resume computes where to restart; the 10:05 pass finishes it automatically in a fresh window.
- **TTS drops mid-stream.** Retried per chunk, then per paragraph; exhausting retries fails that episode only, and the script stays on disk for a manual re-publish.

Cautious Optimism Briefings – agent architecture & flow.

Companion documents: `CLAUDE.md` (the always-on editorial standard) · `docs/ARCHITECTURE.md` (module-level reference) · the `daily-briefing` skill (operational procedure).